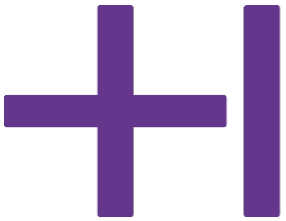
Hochschule Karlsruhe
University of
Applied Sciences

Fakultät für
Informatik und
Wirtschaftsinformatik





Sprint 2

20.10. — 3.11.26

Abgaben für die Sprints jeweils bis 11:30 Uhr in ILIAS

Integration ausgewählter Bausteine



Entwickelt folgende Komponenten, die später im Prozess genutzt werden können:

- Mock REST-API für Retourenmeldungen die, z.B. Von Amazon oder Zalando kommen
- gRPC Service für Wareneingangs- und Zustandserfassung
- Status-Updates zur Retourenbearbeitung über Nachrichtenaustausch

Die Komponenten repräsentieren unterschiedliche Systeme, die wir später einbinden werden.

Die Programmiersprache und -umgebung kann frei gewählt werden.

Verwendet als Basis für den Nachrichtenaustausch einen Message Broker (z.B. ActiveMQ oder RabittMQ).

Weitere Anforderungen und Fragen könnt ihr mit dem Dozenten klären.



© Andreas Heberle

Mock REST-API für Retourenmeldungen



Neue Retourenmeldungen, die bei Amazon oder Zalando eingehen, sollen über einen REST-Service abgerufen werden können.

Es soll möglich sein

- eine Liste alle Retourenmeldungen abzurufen, mit Filtermöglichkeit (z.B. nach Status oder Marktplatz)
- eine einzelne Retourenmeldung über eine *returnID* und die Daten zu einer Bestellung über eine *orderID* abzufragen
- Den Status zu einer Retourenmeldung zu aktualisieren.

Beispiel Retourenmeldung


```
{
 "data": [
  {
   "returnId": "RET-2026-00457",
   "orderId": "ORD-2026-118932",
   "marketplace": "ZALANDO",
   "status": "NEW",
   "reason": "SIZE_TOO_SMALL",
   "reasonText": "Passt leider nicht, eine Nummer kleiner bitte.",
   "requestedAt": "2026-09-18T09:14:00Z",
   "updatedAt": "2026-09-18T09:14:00Z",
   "items": [
    {
     "sku": "SH-BLZ-042-M",
     "productName": "Blazer Slim Fit",
     "quantity": 1,
     "unitPrice": 89.9,
     "currency": "EUR"
    }
   ],
   "customer": {
    "customerId": "CUST-88213",
    "email": "j.mueller@example.com"
   },
   "returnLabelUrl": null
  }
 ]
}
```


Implementiert die API und einen Client, mit dem man Abfragen machen kann.
Die Programmiersprache zur Implementierung des Service steht euch frei.



© Andreas Heberle

gRPC Service zur Wareneingangs- und Zustandserfassung

Der Service bewertet den im Warehouse-Management-System erfassten Zustand eines retournierten Artikels und liefert eine automatisierte Empfehlung: annehmen, ablehnen oder manuell prüfen. Grenzfälle werden später im Workflow manuell geprüft.

- Als Ausgangspunkt für die Implementierung verwendet bitte die mitgelieferte article_condition_service.proto Datei. Nach Rücksprache mit dem Dozenten könnte ihr Anpassungen vornehmen.
- Implementiert mit dem gRPC Framework den Service und einen Client, mit dem man Abfragen an den Service machen kann.
- Der Service simuliert die Bewertung von Retouren und sollte unterschiedliche Prozessszenarien erlauben sowie Fehler berücksichtigen.

Die Programmiersprache zur Implementierung des Service steht euch frei.

© Andreas Heberle

Statusaktualisierungen über Nachrichtenaustausch



Statusaktualisierungen sollen von den beteiligten Systemen per Nachrichtenaustausch mitgeteilt werden. Überlegt dazu, welche sinnvollen Statusübergänge es in dem Prozess geben kann.

Implementiert dann mit Nachrichtenaustausch

- einen Client, der Statusupdates zu Retouren erzeugt (Producer) und
- einen Client, der Statusupdate-Nachrichten zu Retouren abruft (Consumer) und den Status einer Retoure aktualisiert.
- Überlegt, welche Nachrichten Topics/Queues ihr dafür braucht.

Als Message Oriented Middleware könnt ihr ActiveMQ oder RabbitMQ verwenden. Man kann die beide auch mit Docker nutzen.

ActiveMQ unterstützt übrigens die Enterprise Integration Patterns von Gregor Hohpe und Bobby Woolf 😉.

Die Programmiersprache könnt ihr selbst wählen.



© Andreas Heberle

Aspekte Messaging und gRPC



Bewertungsaspekte für Messaging
- saubere Eventdefinition für Payment
- korrekte Producer/Consumer Logik
- Entkopplung der Systeme
- Fehlerhandling
  - (z.B. Nachricht konnte vom Producer nicht ausgeliefert werden)
  - Consumer versteht die Nachricht nicht
  - doppelte Events?
- Gut strukturiertes Coding

Bewertungsaspekte gRPC
- Saubere sinnvolle Schnittstellenbeschreibung
  - Für alle Fälle, die man für die Erfassung von Rechnungen braucht.
- Implementierung mit passenden Exceptions (Service)
  - z.B. doppelte Rechnung
  - Falsche Rechnungsdaten?
- Fehlerhandling Client
  - Doppelte Rechnungen speichern
- Gut strukturiertes Coding



© Andreas Heberle

Hinweise und Literatur



- ActiveMQ Flexible & Powerful Open Source Multi-Protocol Messaging. https://activemq.apache.org/
- G. Hohpe und B. Woolf: Enterprise Integration Patterns. https://www.enterpriseintegrationpatterns.com/
- gRPC A high performance, open source universal RPC framework. grpc.io
- Rabbit MQ Broker. https://www.rabbitmq.com/

Für die Implementierung sollt ihr die Standards und Best Practices anwenden, die ihr bei Prof. Zimmermann gelernt habt.

Ihr dürft KI für die Bearbeitung der Aufgabe verwenden, aber

- Denkt nach und versteht, was ihr produziert und ob das in eurem Kontext Sinn macht.
- Ich werde dazu natürlich Fragen stellen 😉.
- Und ich will dokumentiert haben, was mit KI erstellt wurde.



© Andreas Heberle